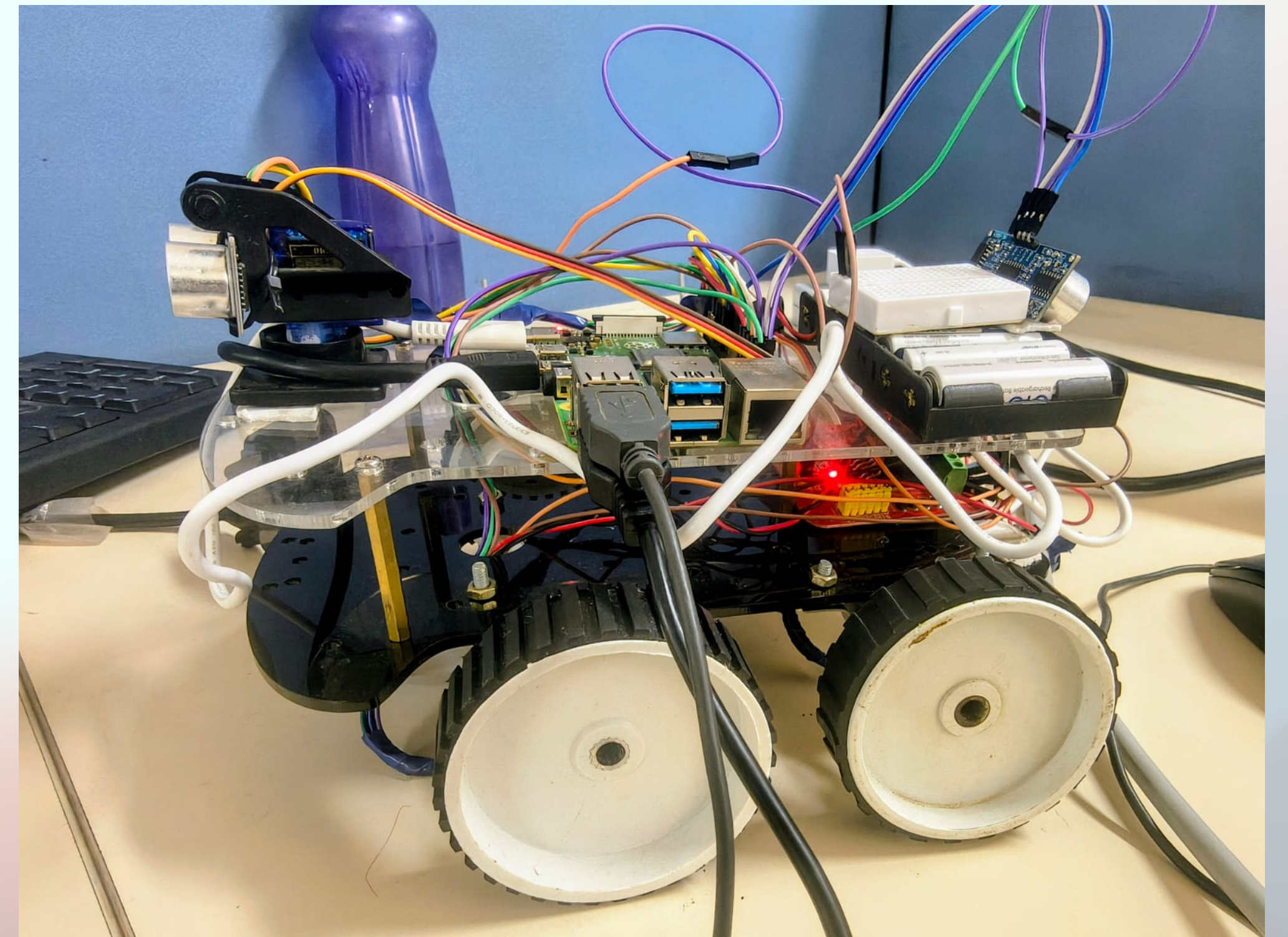


ML Equivalent Control Strategy For ACC Considering Front And Rear Vehicles

Valupadasu Revanth | 25AT63R04

CPS Application Lab

Guided by: Dr. Somnath Sengupta



OBJECTIVES & IMPORTANCE OF THE PROBLEM

Problem Statement

Traditional ACC systems rely on classical PID controllers that require manual tuning for every environment. This project builds a miniature testbed to investigate ML-enhanced predictive control—combining MPC with trained MLP networks—that adapts automatically and is aware of both front and rear traffic.

Minimum PWM Discovery

Find exact threshold PWM via GUI-controlled car testing to overcome motor deadband

ML-Based Speed Control

Replace traditional PID with a NumPy MLP model trained on real telemetry data

Front-Aware MPC

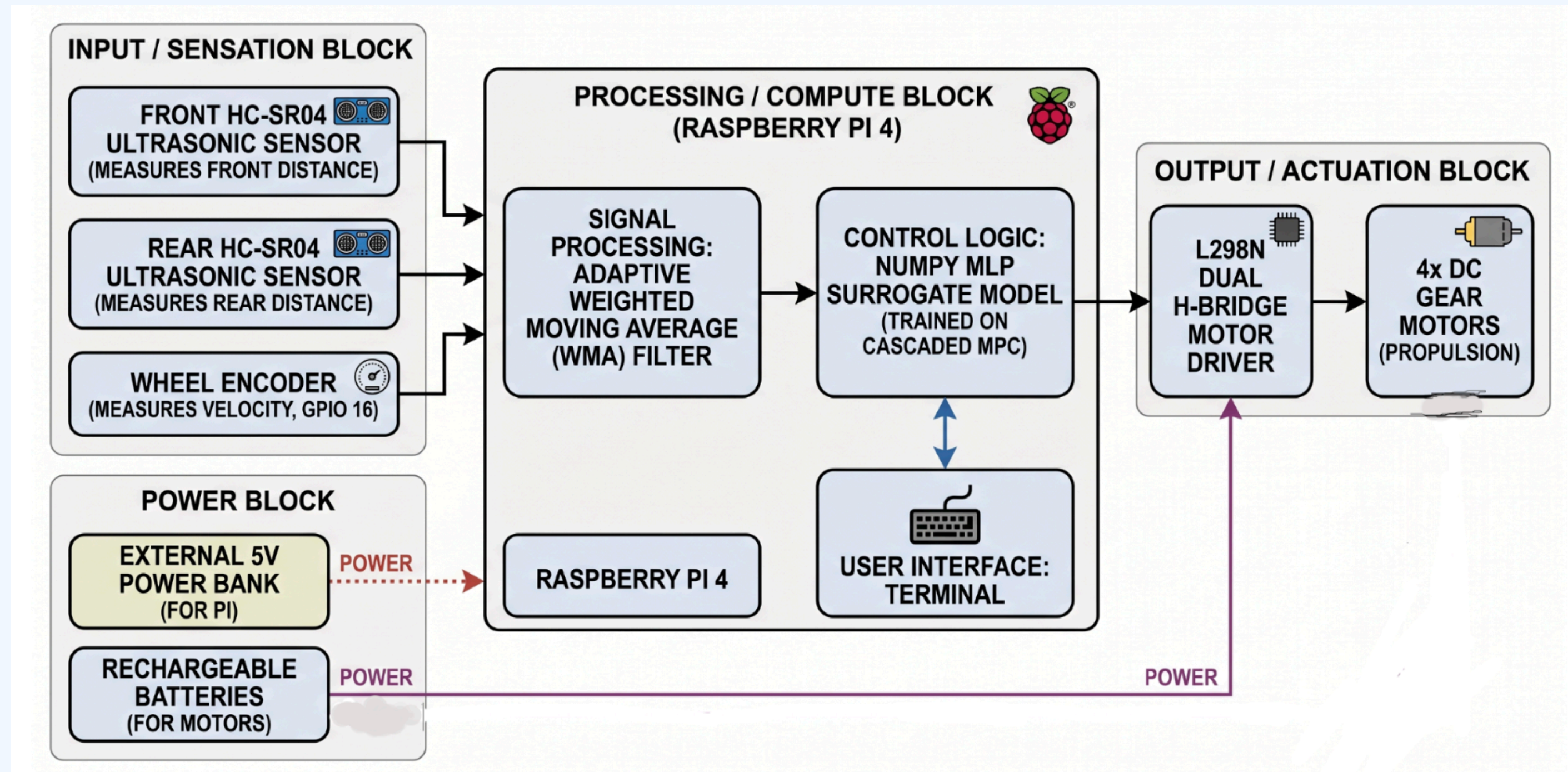
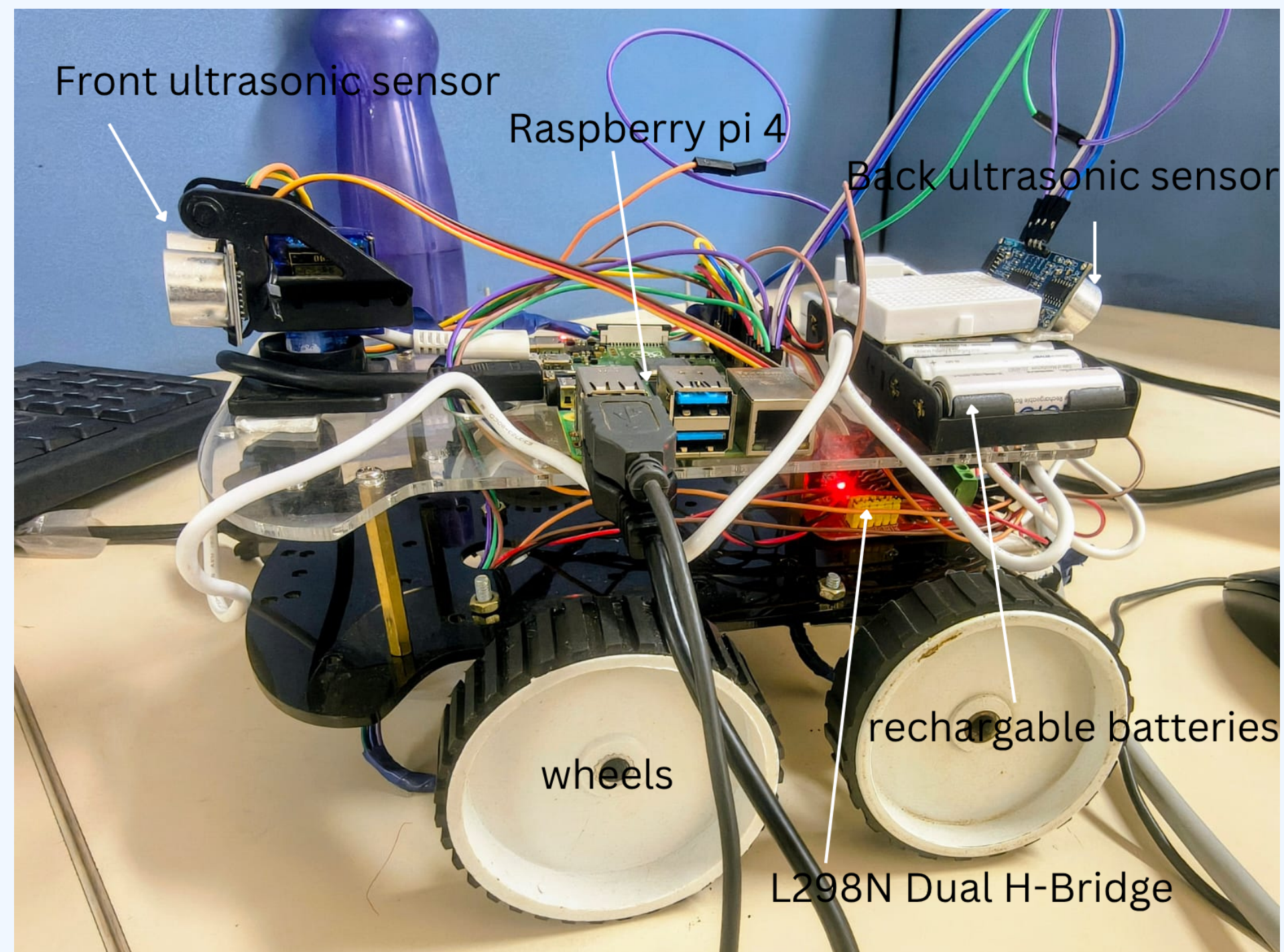
Simulate & train MLP to replicate cascaded MPC behaviour for front vehicle following

Dual-Aware ACC

Extend to dual ultrasonic sensing (front+rear) with Hildreth QP-solved MPC1+MPC2

INTRODUCTION & BACKGROUND THEORY

Classical Adaptive Cruise Control requires manual tuning and ignores rear traffic. This project tackles these limitations through a phased approach to embedded machine learning control. From developing an ML-based PID controller, to a Front-Aware MPC surrogate, and building a fully integrated Dual-Aware MPC system.



IMPLEMENTATION: HARDWARE & SOFTWARE ARCHITECTURE

⚙️ Hardware Components

Raspberry Pi 4

Main compute unit running Python controller

L298N Motor Driver

Dual H-bridge

HC-SR04 (×2)

Front (GPIO23/24) & Rear (GPIO20/21) sensors

Encoder

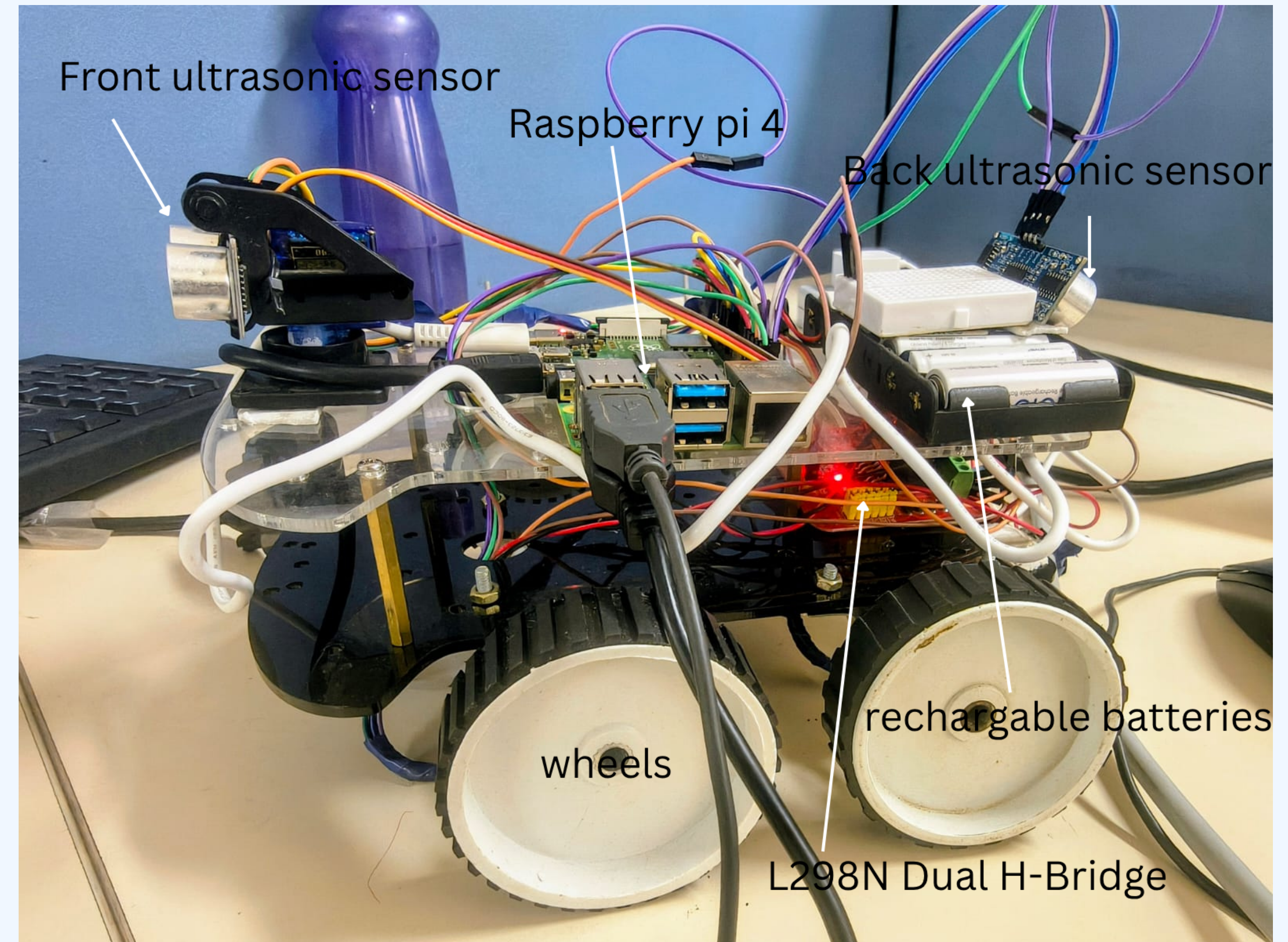
GPIO16 – 348 counts/rev, 65mm wheel ($\pi \times 0.065\text{m}$)

DC Motors (×4)

Min deadband PWM $\approx 95/255$ found via GUI test

power bank and rechargeable batteries

Power supply



IMPLEMENTATION: HARDWARE & SOFTWARE ARCHITECTURE

⚙️ software Components

Python

for the controller logic, inference, and the GUI

Libraries & Frameworks

NumPy

used to run the trained Multilayer Perceptron (MLP) surrogate models with sub-millisecond inference speeds

TensorFlow

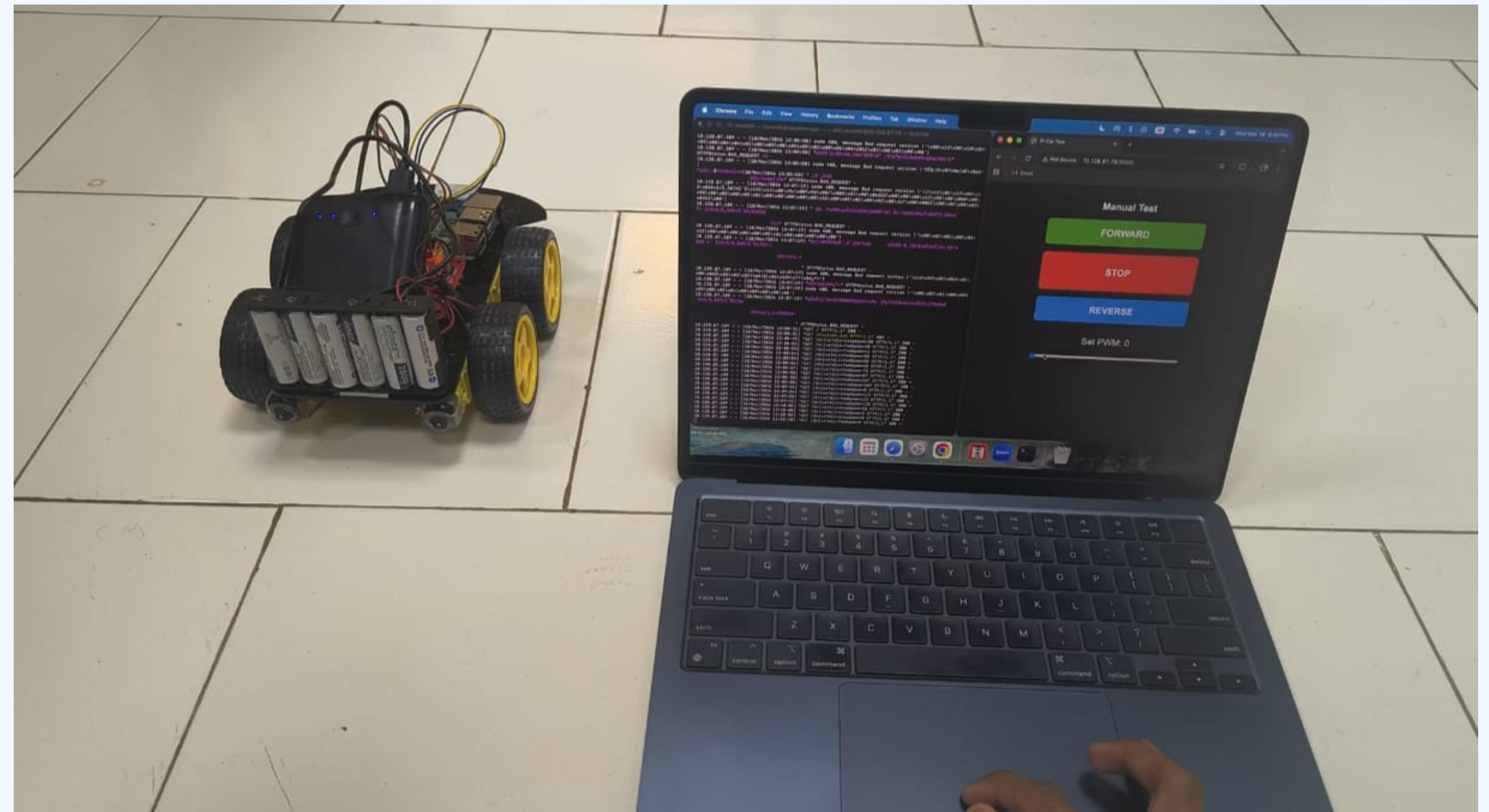
Initially used for the baseline PID controller and off-board model training. It was intentionally removed from the final deployment on the Raspberry Pi due to environment incompatibility and import errors.

Flask

browser-based GUI interface

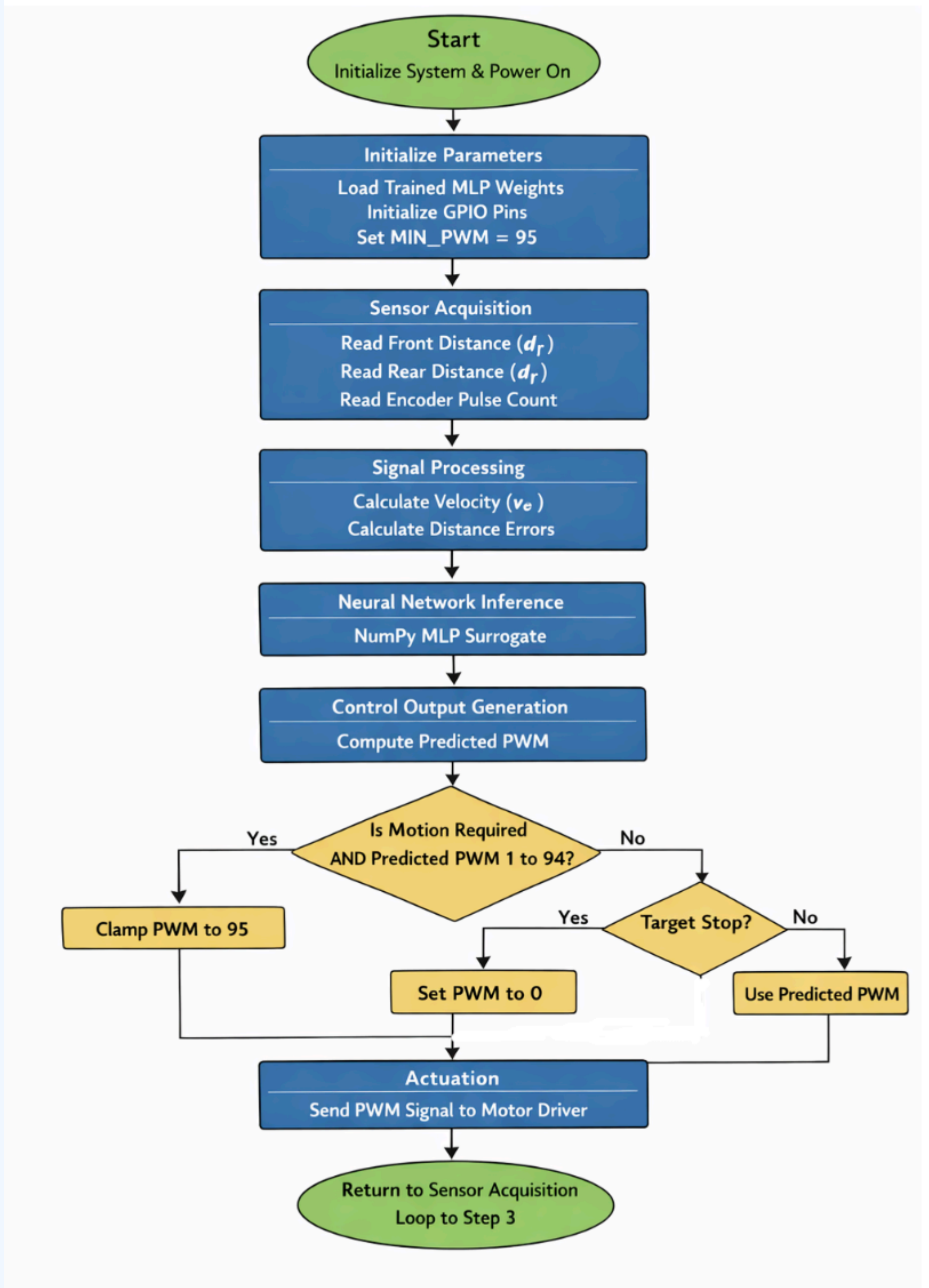
Tools : Google Colab

To train the model



IMPLEMENTATION

FLOWCHART:



Control Pipeline

Phase 1: GUI Calibration

Flask server, PWM 0–255 slider to find minimum effective PWM ≈ 95

Phase 2: ML-PID (TensorFlow to NumPy)

MLP with 2 hidden layers trained on (4 features $\rightarrow$ PWM output)

Phase 3: Front MPC

8000-point CSV simulation $\rightarrow$ MLP trained to mimic cascaded MPC (RMSE optimized)

Phase 4: Dual MPC

HighLevelMPC (Hildreth QP, 1-state kinematic) + LowLevelMPC (3x3 dynamics model tracking)

Phase 5: ML Dual Test

MLP deployed on-car with dual sensors; rear-approach triggers speed increase

IMPLEMENTATION: Working mechanism

ML-PID

```
features = [  
    'distance_cm',  
    'safe_distance_cm',  
    'target_velocity_cmps',  
    'actual_velocity_cmps']  
  
X = df[features]  
y = df['pwm_output']
```

Front-Aware MPC

```
▶ features = [  
    'distance_error_cm',  
    'velocity_error_cmps',  
    'actual_velocity_cmps',  
    'leader_velocity_cmps',  
    'prev_pwm_output']  
  
X = df[features]  
y = df['pwm_output']
```

Dual-Aware MPC

```
features = ['ve',  
            'vf',  
            'vr',  
            'df',  
            'dr',  
            'd_safe_f',  
            'd_safe_r',  
            'delta_df']  
  
X = df[features]  
y = df['u_total']
```

Training the Model:

```
▶ model = MLPRegressor(  
    hidden_layer_sizes=(64, 32),  
    activation='relu',  
    solver='adam',  
    alpha=0.001,  
    max_iter=3000,  
    random_state=42  
)
```


ML-PID

```
features = [  
    'distance_cm',  
    'safe_distance_cm',  
    'target_velocity_cmps',  
    'actual_velocity_cmps']  
  
X = df[features]  
y = df['pwm_output']
```

distance_cm (how far away the object is),
safe_distance_cm (how far away it *should* be),
target_velocity_cmps (how fast it wants to go),
actual_velocity_cmps (how fast it is actually going).

distance_error_cm (the difference between safe and actual distance)
velocity_error_cmps (difference between how fast your car *should* be going and how fast it is *actually* going.).
actual_velocity_cmps: This is the real-time speed of your robotic car.
It also looks at the speed of the car in front (leader_velocity_cmps)
remembers what it just did a millisecond ago (prev_pwm_output).

Front-Aware MPC

```
features = [  
    'distance_error_cm',  
    'velocity_error_cmps',  
    'actual_velocity_cmps',  
    'leader_velocity_cmps',  
    'prev_pwm_output'  
]  
  
X = df[features]  
y = df['pwm_output']
```

Dual-Aware MPC

```
features = ['ve',  
            'vf',  
            'vr',  
            'df',  
            'dr',  
            'd_safe_f',  
            'd_safe_r',  
            'delta_df']  
  
X = df[features]  
y = df['u_total']
```

It looks at its own velocity (ve),
the front car's velocity (vf),
the rear car's velocity (vr),
the distance to the front car (df),
the distance to the rear car (dr),
and what the safe distances should be for both (d_safe_f, d_safe_r).
delta_df stands for Front Distance Error.

Training the Model:

```
▶ model = MLPRegressor(  
    hidden_layer_sizes=(64, 32),  
    activation='relu',  
    solver='adam',  
    alpha=0.001,  
    max_iter=3000,  
    random_state=42  
)
```

MLPRegressor (specifically in Python libraries like scikit-learn) is an exact, ready-to-use implementation of an MLP that is **hard-wired to output a continuous numerical value**.

The "Regressor" part is the key. In machine learning, *regression* means predicting a sliding scale of numbers rather than distinct categories.

hidden_layer_sizes=(64, 32): This is the size of the AI's "brain."

activation='relu': This is the math function that helps the network understand complex, non-straight-line relationships.

solver='adam': This is the specific mathematical strategy the AI uses to learn from its mistakes and get better over time.

max_iter=3000: This means you let the AI practice the data up to 3,000 times to learn the patterns.

RESULTS & PERFORMANCE ANALYSIS

Key Performance Metrics Summary:

Scenario	Inference (ms)	R ²	RMSE	MAE	Train Iters	SAMPLES
Dual Sim	0.145	0.947	0.036	0.022	168	8000
Dual Speed	—	—	0.186	0.116	106	2558
Front Real	0.398	0.988	7.203	1.600	372	2540
Front Sim	0.179	0.991	7.742	2.919	494	8000
PID Real	0.138	0.946	14.637	8.186	1770	935
PID Sim	0.316	0.994	6.452	3.618	301	8000

Dual model uses normalized data (-1 to 1) and the PID uses raw PWM (0-255)

RESULTS & PERFORMANCE ANALYSIS

Inference (ms): Reaction time. This is how many milliseconds it takes the Raspberry Pi to look at the sensors and decide on a motor command. Lower is better.

R² (R-squared): The "Copycat Score." This measures how perfectly your AI cloned the heavy mathematical equations of the original controllers. A perfect score is 1.0.

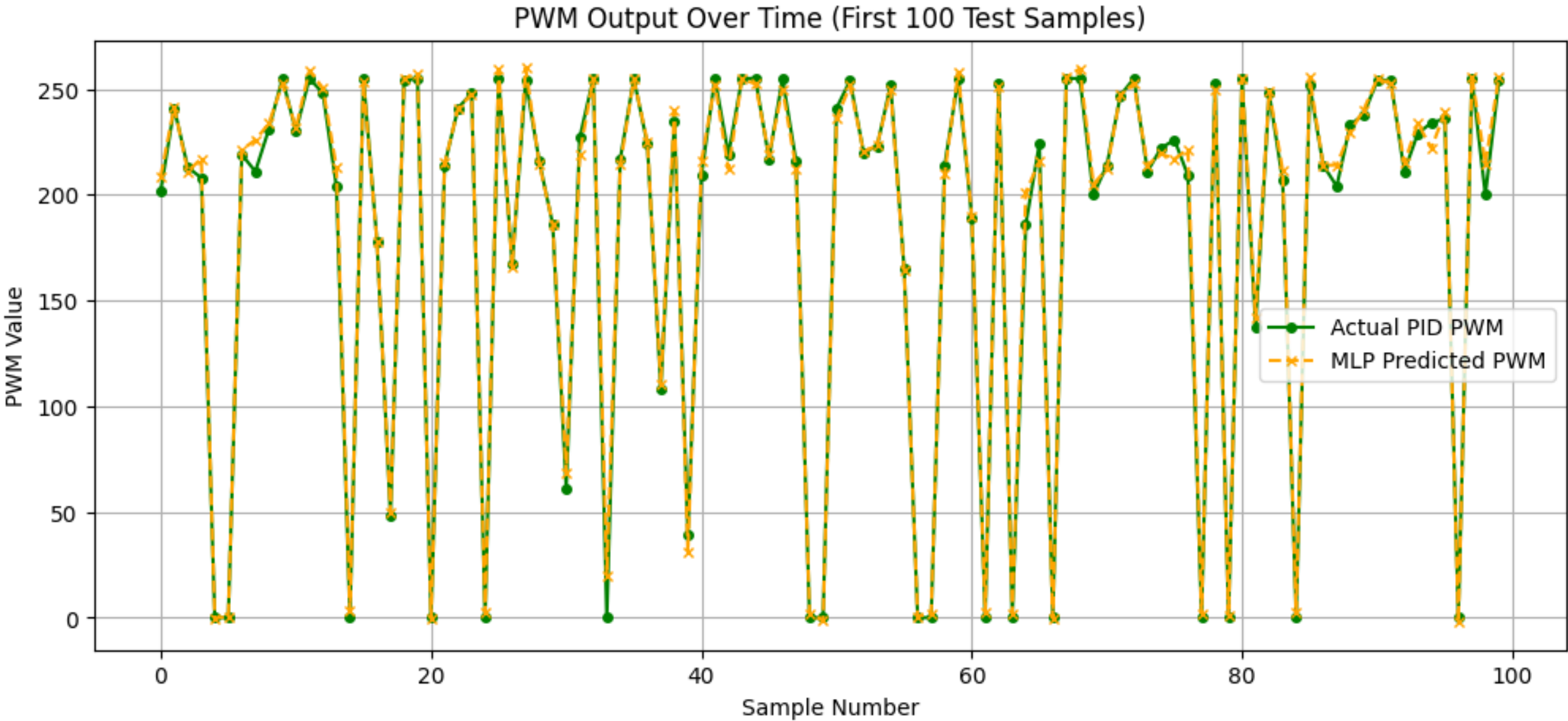
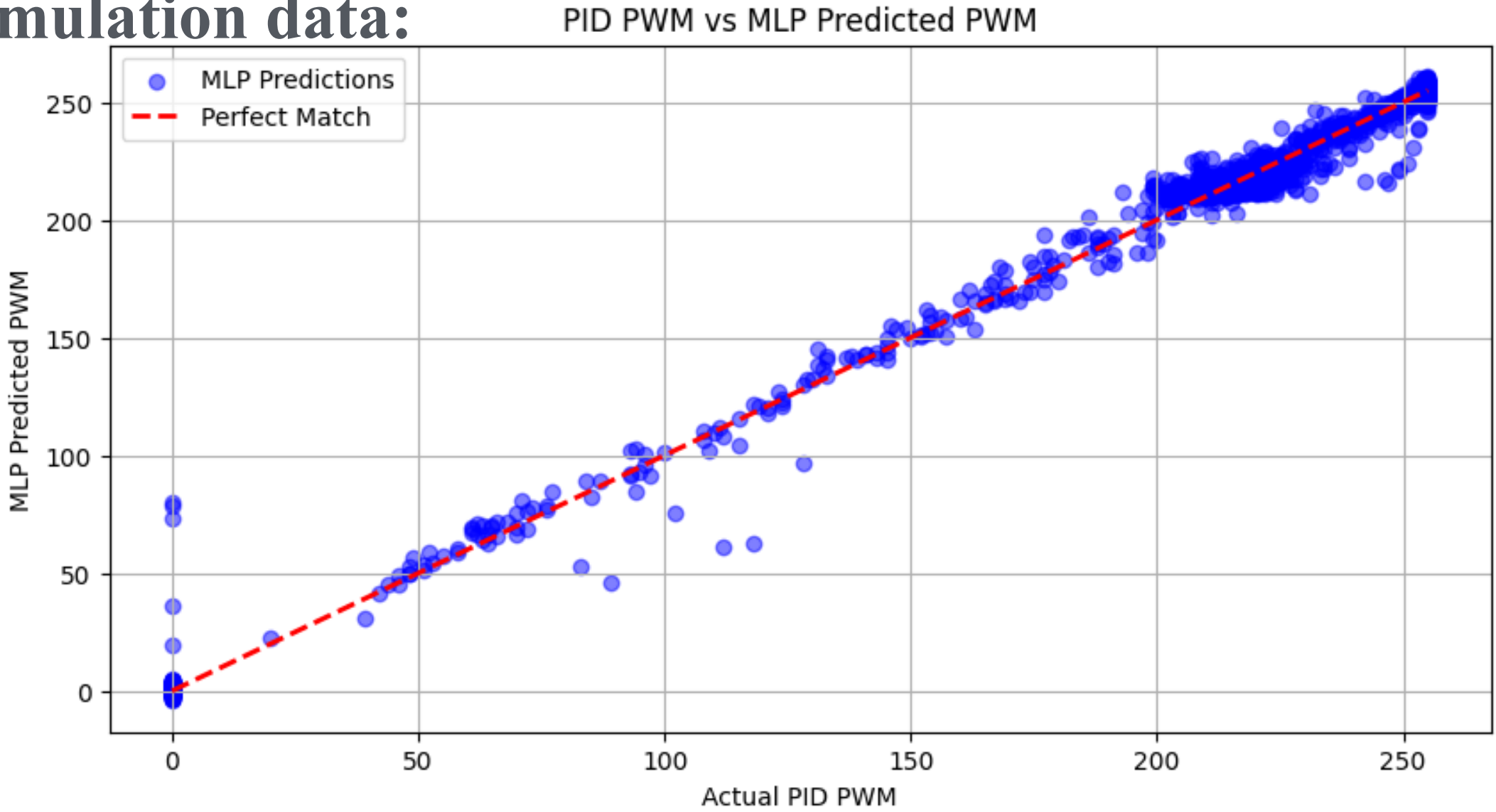
RMSE & MAE: The "Mistake Trackers." These measure the average error between what the AI guessed the motor power should be and what the perfect math said it should be. Lower is better. *(Note: The "Dual" models output a small number between -1 and 1, while the others output a PWM between 0 and 255. That is why the Dual errors look so much smaller).*

Train Iters: How many "practice loops" the AI went through before it figured out the patterns and stopped training.

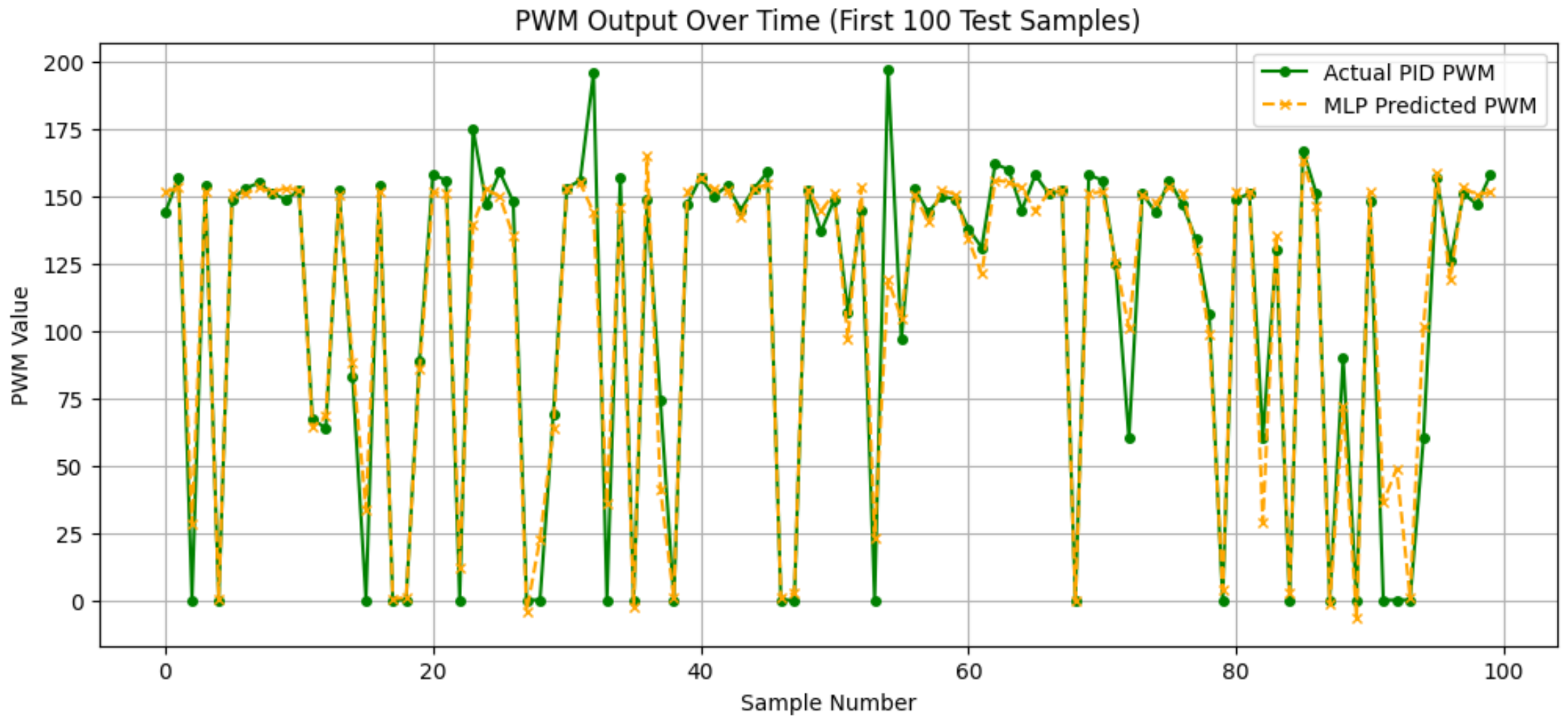
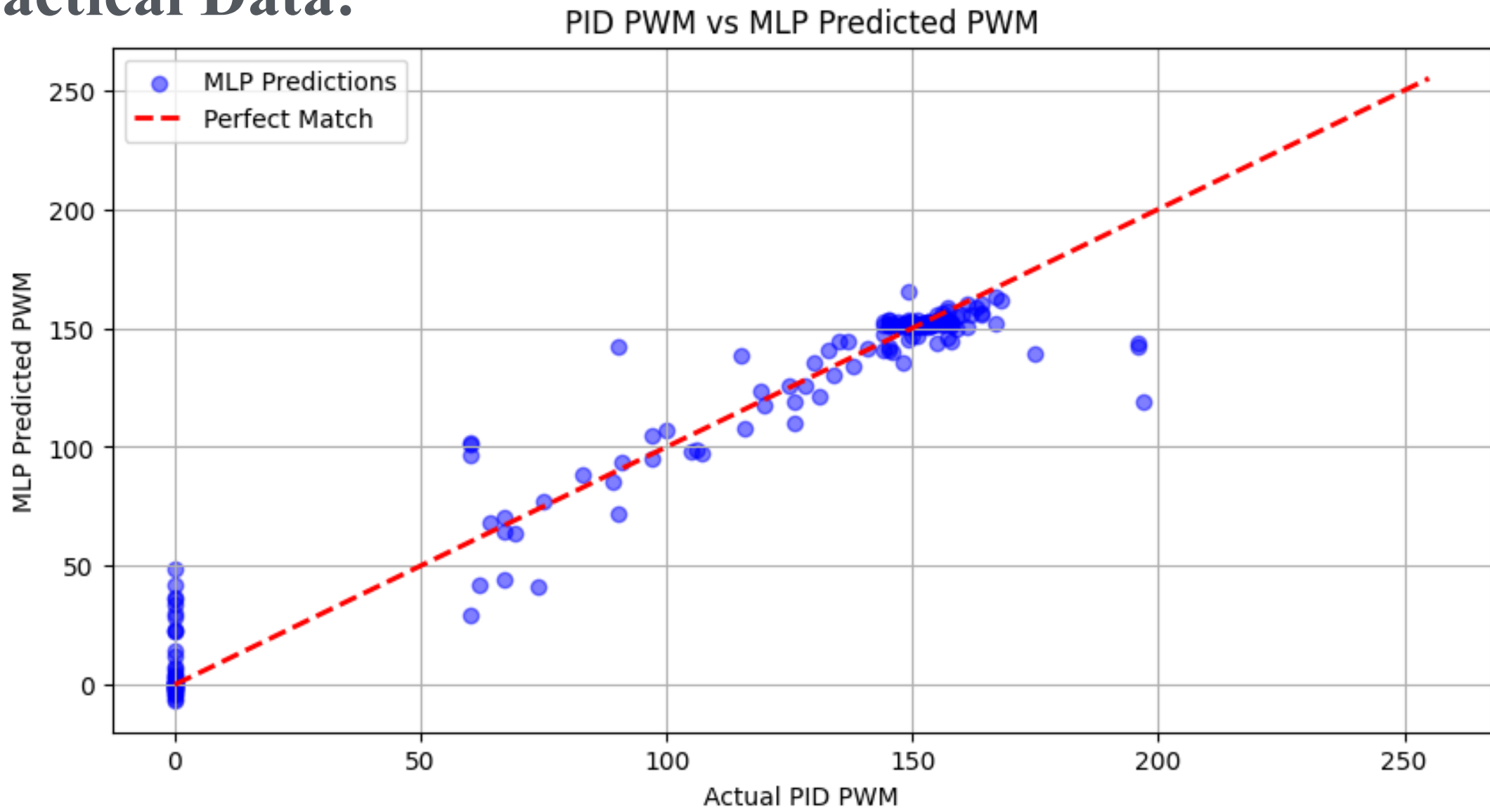
SAMPLES: The number of data points (like flashcards) you fed the AI to teach it.

RESULTS & PERFORMANCE ANALYSIS(trained using PID data)

simulation data:

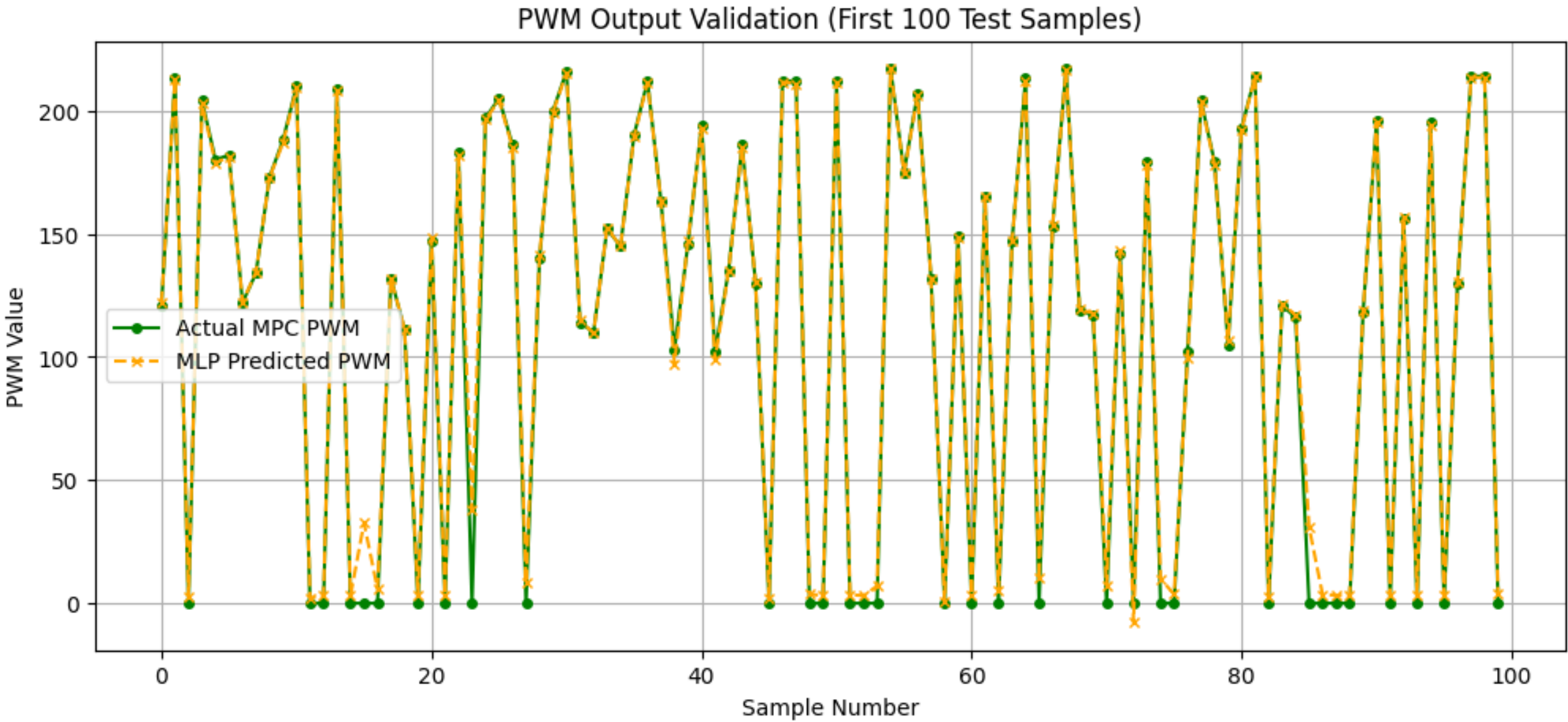
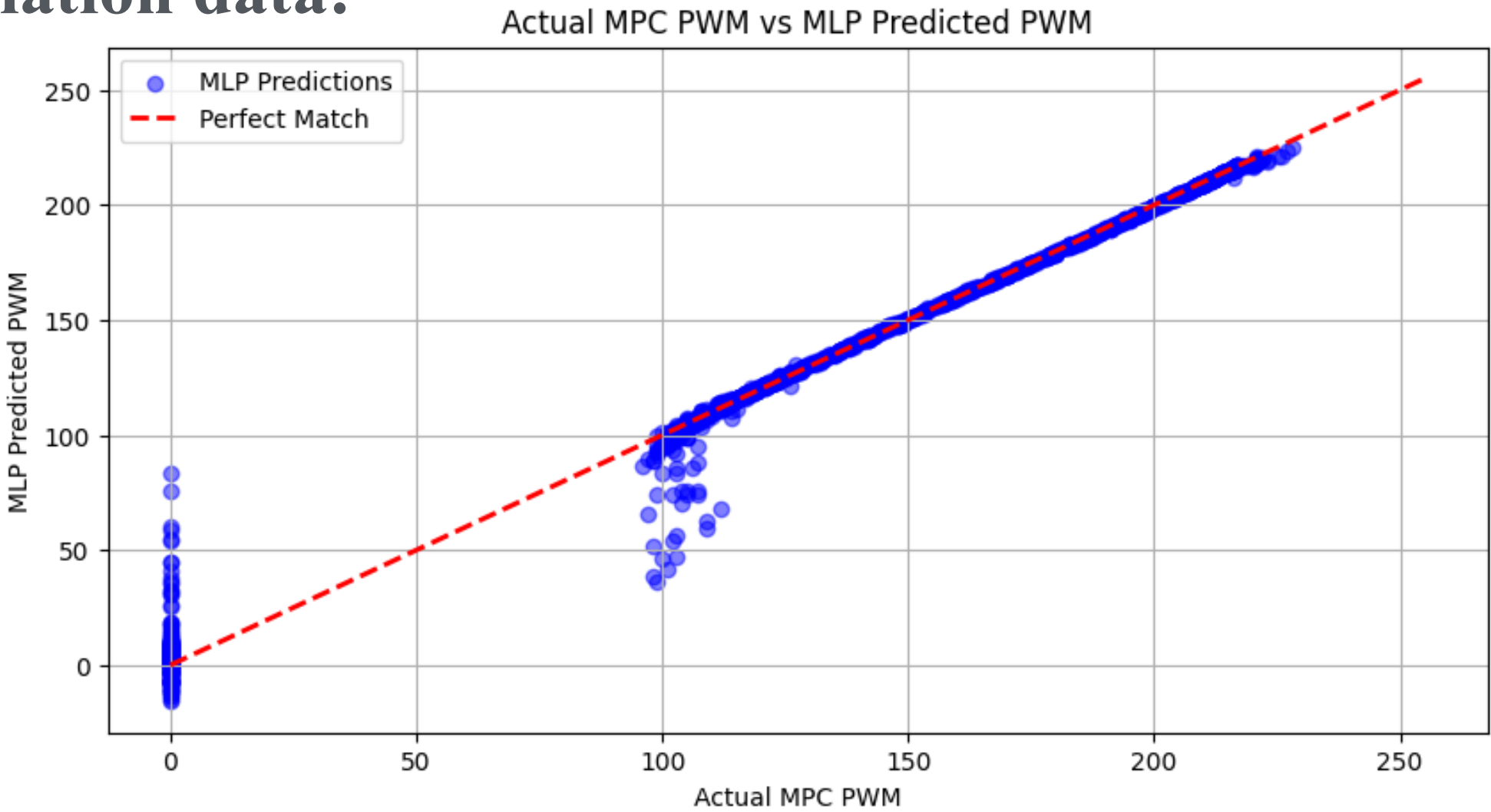


Practical Data:

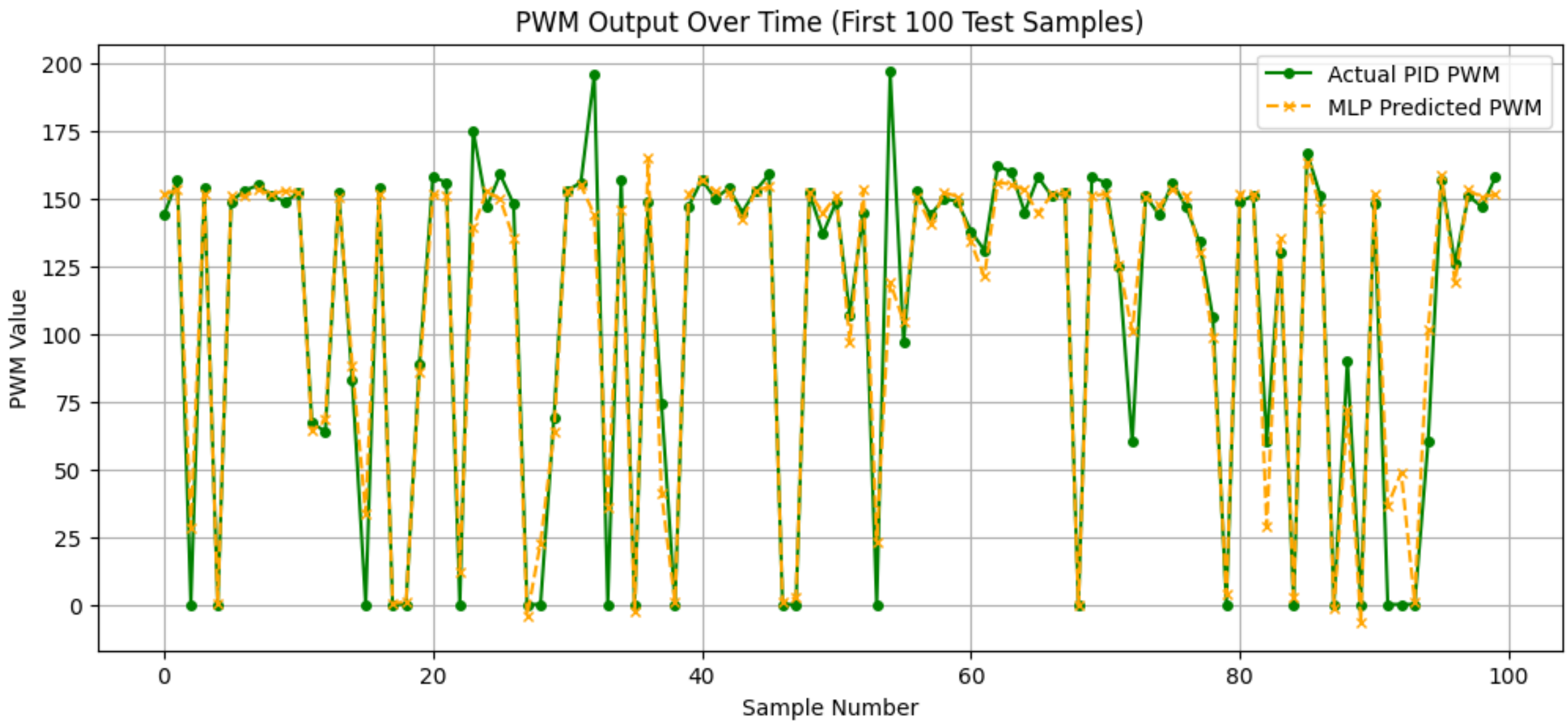
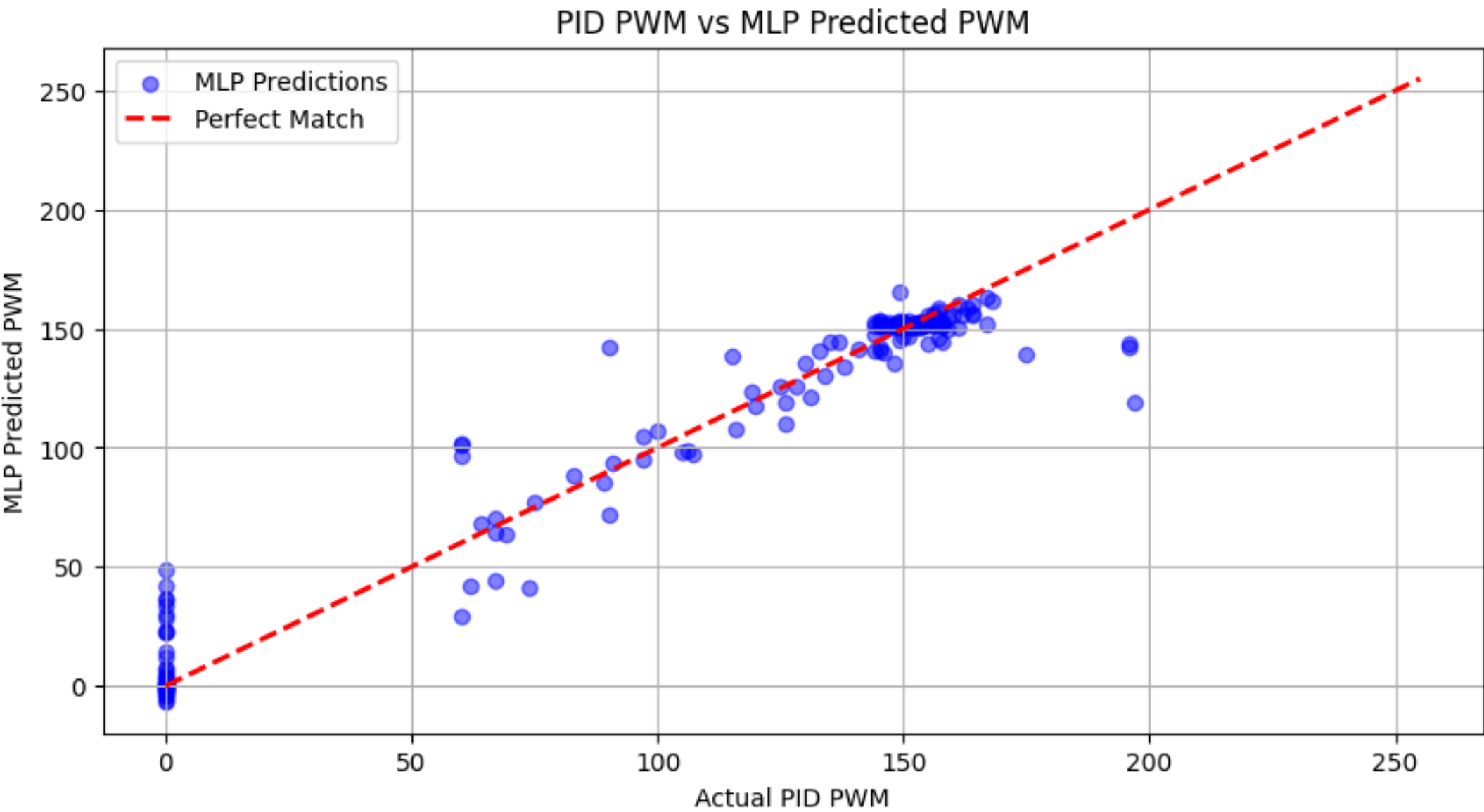


RESULTS & PERFORMANCE ANALYSIS(trained using front aware mpc data)

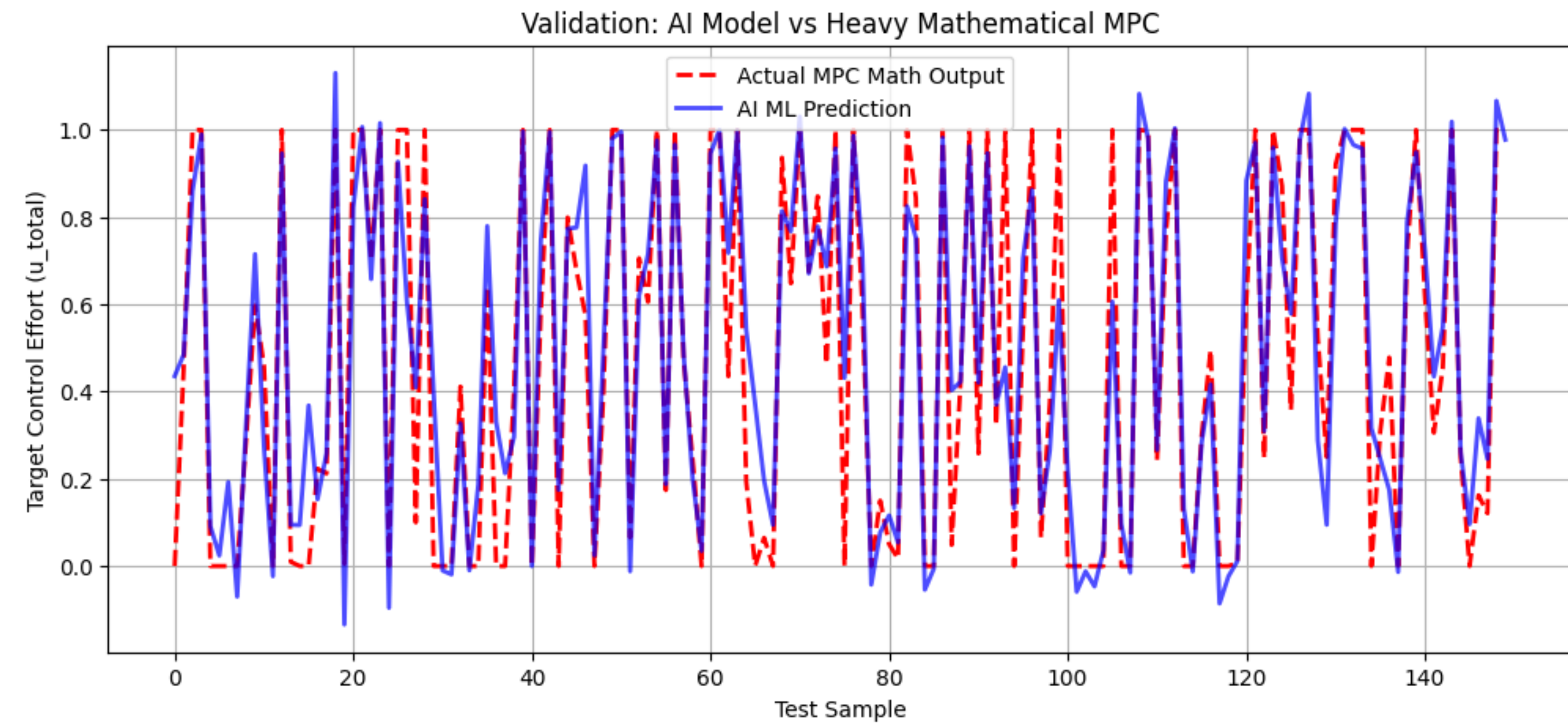
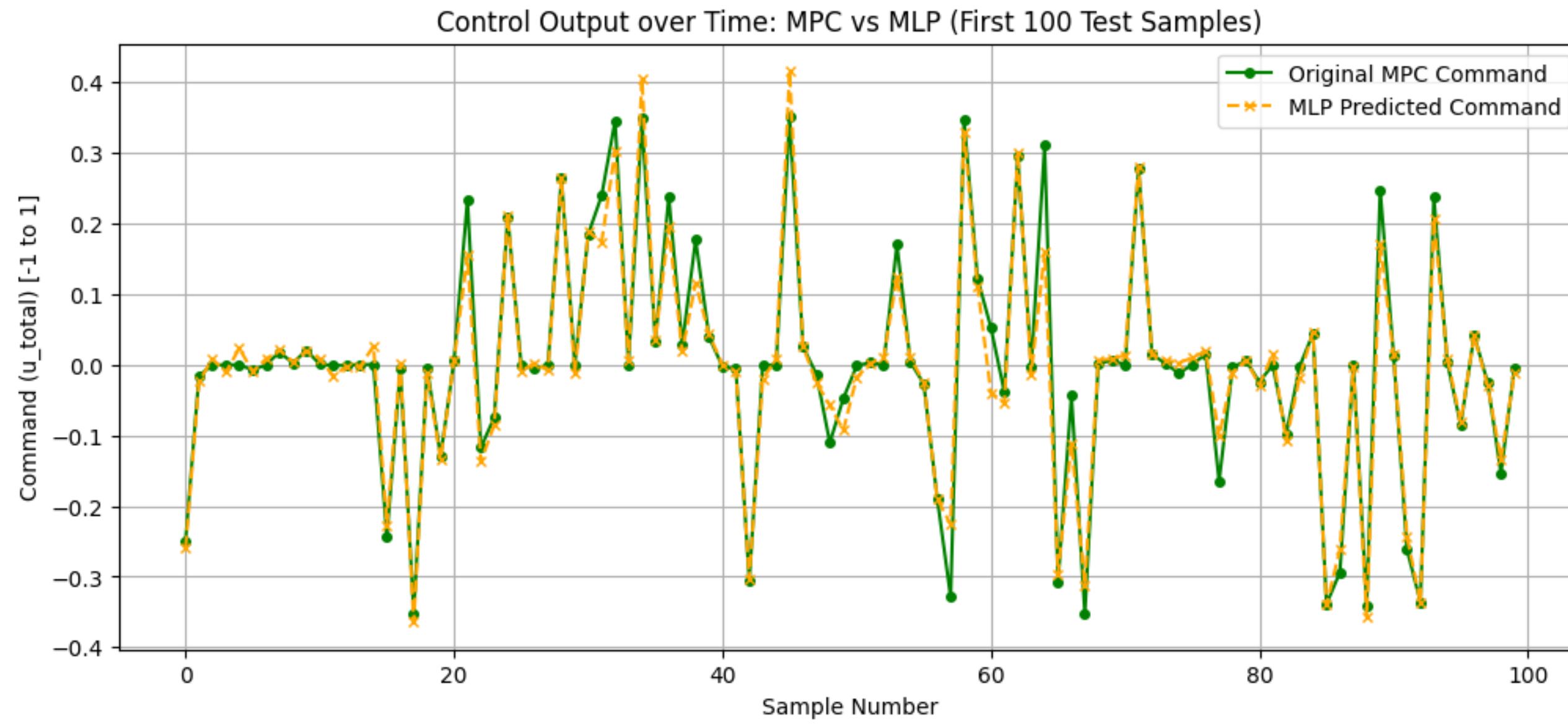
simulation data:



Practical Data:



RESULTS & PERFORMANCE ANALYSIS(dual aware)



CHALLENGES FACED & MITIGATION EFFORTS

01

TensorFlow Not Running on Car

⚠ The ML-based PID originally from reference code used TensorFlow, which caused import errors and incompatibility on the Raspberry Pi external environment.

✓ Completely rewrote the inference pipeline using only NumPy. Extracted weights from trained model into .npz files. Full forward pass (ReLU activations) reimplemented manually.

02

Motor Deadband / No Movement at Low PWM

⚠ Car would not move below a certain PWM value. Classical control assumed linear response from 0–255.

✓ Built Flask-based web GUI (frontvehiclewithoutus.py) with live PWM slider. Empirically determined min effective PWM $\approx 95/255$. Hard-coded as MIN_PWM in all controllers.

03

Dual-Aware MPC Fine-Tuning Incomplete

⚠ The cascaded dual MPC (dualaware.py) compiles and runs but optimal Q/R weight values for the Hildreth QP have not been experimentally validated on hardware.

✓ Simulation confirmed correct architecture. mldual.py deploys a trained MLP on-car as interim; rear-awareness logic triggers speed increase when rear vehicle approaches.

04

ML Dual-Aware Ignores Set-Speed

⚠ The current mldual.py model only reacts to proximity events (front/rear) but does not maintain the configured cruise speed properly.

✓ Identified root cause: training data lacked sufficient cruise-phase examples. Adding v_set as explicit input feature and re-training with balanced dataset is done.

05

Encoder Velocity Noise & Spike Rejection

⚠ Wheel encoder gave spurious high-velocity spikes under vibration, corrupting the filtered velocity estimate.

✓ Implemented adaptive WMA filter with spike rejection (>0.3 m/s change in one step ignored). Separate α values for different error magnitudes (0.25–0.70).

CONCLUSION & FUTURE SCOPE

What We Achieved

Built a working miniature ACC testbed on Raspberry Pi with dual ultrasonic sensors, encoder feedback, and L298N motor driver

Identified minimum PWM deadband through systematic GUI-based hardware calibration

Successfully migrated ML inference from TensorFlow to NumPy-only MLP — achieving full on-device deployment

Reduced velocity tracking RMSE by ~50% over baseline TF-PID through improved architecture and training data

Designed and implemented a complete dual-aware cascaded MPC (MPC1+MPC2) with Hildreth QP solver

Deployed ML-based dual-aware controller on hardware; rear-awareness principle validated

Future Scope

Complete Dual MPC Tuning

Fine-tune Q/R weights experimentally on hardware for optimal front+rear tracking

Reinforcement Learning Policy

Transitioning from supervised MLP mimicry to an RL agent

RTOS Integration

To guarantee hard real-time execution and eliminate control loop jitter

Sensor Fusion Pipeline

Upgrading from raw ultrasonic data to a robust sensor fusion architecture using Extended Kalman Filters (EKF) to handle sensor noise and dropouts

Multi-Vehicle Platooning

Extend to 3+ vehicle platoon with V2V communication simulation